

## 7 Interrupts

### 7.1 Beschrijving

Een interrupt is een verzoek aan de processor om onmiddellijk zijn hoofdprogramma te verlaten en een interrupt routine af te handelen. Deze **interrupt handler** verwerkt de situatie, waarna de processor opnieuw met zijn hoofdprogramma kan verder gaan.

Vlak vóór het verlaten van het hoofdprogramma worden de huidige **program counter** en een aantal andere essentiële processor registers, **weggeschreven naar de stack**. Daarna wordt de program counter ingesteld op het adres van de interrupt (**vector table**) om zo naar de **interrupt handler te verwijzen**.

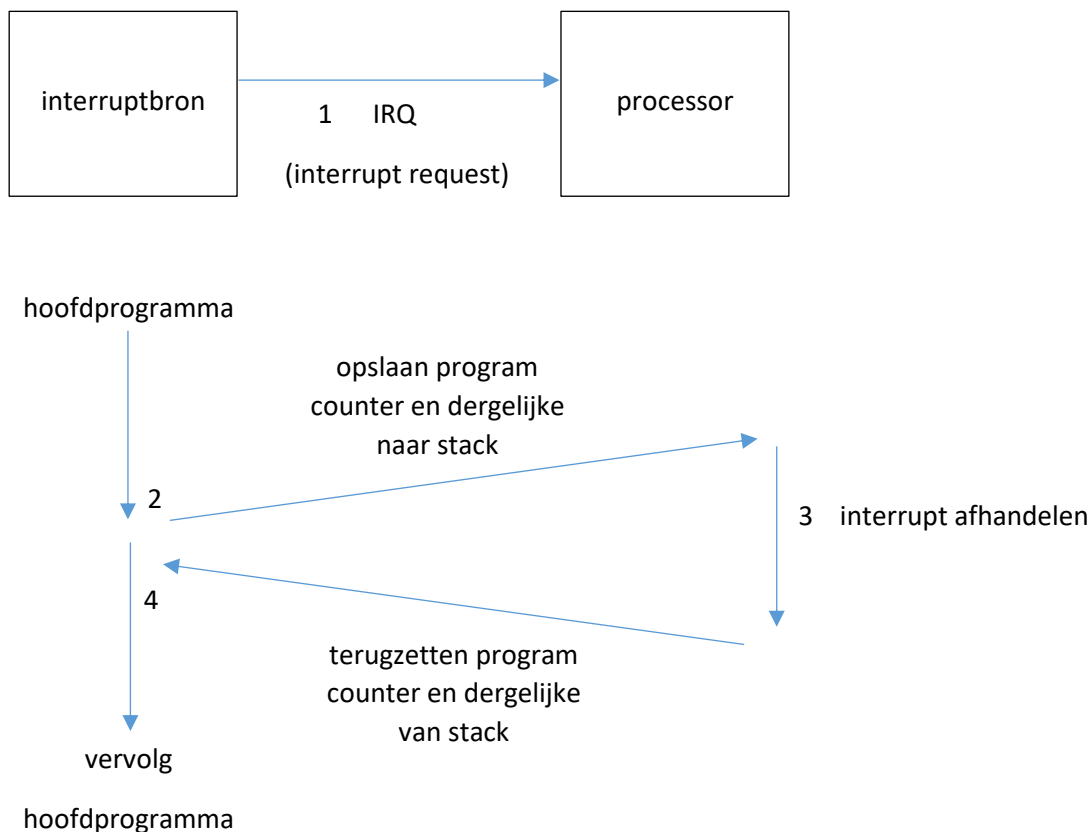
- Program counter: microprocessor register dat het adres van de volgende instructie bijhoudt.
- Stack: geheugenstructuur die wordt aangemaakt in het RAM-geheugen per uitvoerende/actieve functie in de code om tijdelijk gegevens, zoals registerwaarden en return adressen, op te slaan.

Na het afhandelen van de interrupt worden de gegevens van de stack teruggezet zoals voorheen en kan het **hoofdprogramma** verdergaan zoals dit het geval was vóór de interrupt.

Er zijn verschillende bronnen van interrupts, ze kunnen zowel softwarematig als hardwarematig zijn. Vooral periferie kunnen interrupts veroorzaken.

Het voordeel van het werken met interrupts is dat de aanvrager van de interrupt heel snel op zijn wenken bediend wordt!

Het **interruptconcept** kan als volgt voorgesteld worden:



*Figuur 30: visuele voorstelling van het interruptconcept*

## 7.2 NVIC

De **NVIC** (Nested Vectored Interrupt Controller) is de schakel tussen de interruptbron en de CPU en kan 32 interruptkanalen in goede banen leiden. Daarbovenop zijn er nog 16 Cortex M0-interruptlijnen.

### 7.2.1 Prioriteitsniveaus

Die 32 interruptkanalen kunnen volgens **4 programmeerbare prioriteitsniveaus** (0 tot 3) afgehandeld worden. Als er zich op het zelfde moment twee interrupts voordoen die dezelfde programmeerbare prioriteit hebben meegekregen, wordt eerst de interrupt afgehandeld van diegene met de **laagste vaste prioriteit** in de vector table (zie reference manual datasheet p. 218).

Voorbeeld met *EXTIO\_1* en *SPI1*: als er op hetzelfde moment een interrupt binnenkomt van *EXTIO\_1* (vaste prioriteit 12 en programmacode adres 0x0000 0054) en *SPI1* (vaste prioriteit 32 en programmacode adres 0x0000 00A4), waarvan de programmeerbare interruptprioriteit beide op 0 stonden, dan wordt eerst de *EXTIO\_1* interrupt afgehandeld en daarna de *SPI1* interrupt.

In code wordt dit:

```
NVIC_SetPriority(EXTIO_1_IRQn, 0);

NVIC_SetPriority(SPI1_IRQn, 0);
```

*Figuur 31: programmeerbare interruptprioriteiten gelijk zetten*

Opmerking: voorgaande code hoeven we eigenlijk niet expliciet te schrijven vermits de programmeerbare interruptprioriteit standaard steeds op 0 staat.

Als we om programmatorische redenen toch eerst de interrupt van *SPI1* willen afhandelen, kunnen we via de programmeerbare interruptprioriteit de *SPI1* een lagere prioriteit instellen:

```
NVIC_SetPriority(EXTIO_1_IRQn, 1);

NVIC_SetPriority(SPI1_IRQn, 0);
```

*Figuur 32: programmeerbare interruptprioriteiten verschillend maken*

### 7.2.2 Vector mapping

Bij de opstart van de microcontroller moet er gereset worden en naar de *main*-functie gegaan worden. Om dat correct te laten verlopen moet de Vector Mapping goed zijn, ook voor de interrupts wordt die *mapping* belangrijk. Bekijk eens het Assembly-bestand *startup\_stm32f091xc.s* in het Keil-project onder *Device*.

|                        |                  |                                |                                   |
|------------------------|------------------|--------------------------------|-----------------------------------|
| <code>__Vectors</code> | <code>DCD</code> | <code>__initial_sp</code>      | <code>; Top of Stack</code>       |
|                        | <code>DCD</code> | <code>Reset_Handler</code>     | <code>; Reset Handler</code>      |
|                        | <code>DCD</code> | <code>NMI_Handler</code>       | <code>; NMI Handler</code>        |
|                        | <code>DCD</code> | <code>HardFault_Handler</code> | <code>; Hard Fault Handler</code> |
|                        | <code>DCD</code> | <code>0</code>                 | <code>; Reserved</code>           |

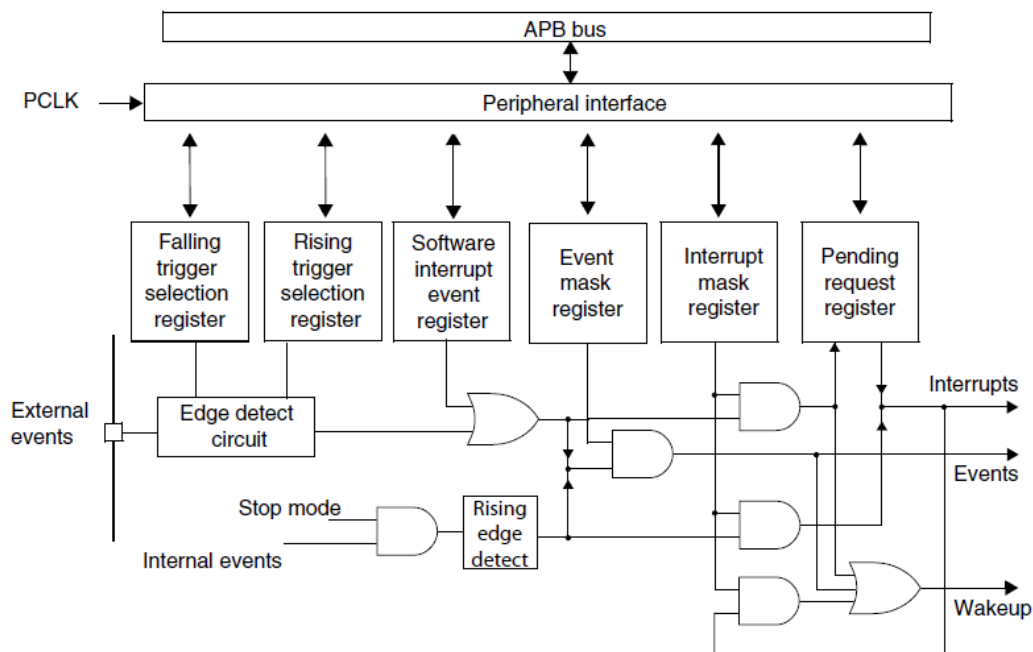
*Figuur 33: codefragment startup\_stm32f091xc.s*

Opmerking: DCD (Define Constant Data) is een Assembly keyword waarmee we één of meerdere 32-bits (4 bytes) woorden in het geheugen kunnen reserveren en initialiseren met specifieke waarden.

### 7.3 Externe interrupts

Eén type van interrupts die de processor kan verwerken, komt van de EXTERNAL Interrupt and event controller. Via deze weg kunnen we gemakkelijk interrupts van 32 bronnen (23 externe en 9 interne) tot bij de processor loodsen.

Het prinsipeschema ziet er als volgt uit:



Figuur 34: blokschema externe interrupt en event controller

#### Vragen:

- Bespreek de verschillende blokken.
- Teken de weg die gevolgd wordt om een stijgende flank op een pin tot bij de core voor een externe interrupt te brengen.

### 7.4 Andere interruptbronnen

De meeste periferie hebben de mogelijkheid om één of meerdere type interrupts in te stellen. We maken onderscheid tussen **externe** interrupts (zoals stijgende of dalende flank op een GPIO-pin) en **interne** interrupts (zoals aflopende timer, afgeronde AD-conversie, ...).

Een aantal hiervan komen verder aan bod in deze cursus.

Telkens wanneer we een interrupt willen gebruiken, moeten alle betrokken registers correct worden ingesteld. Het is raadzaam de reference manual datasheet bij de hand te houden voor de juiste instellingen.

## 7.5 Instellingen externe interrupts

We kunnen de externe interrupts gebruiken om via een knop een interrupt te genereren. Hieronder staat de code om de interrupt in te stellen voor SW2 op het Nucleo Extension Shield:

```
// SYSCFG clock enable.
RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;

// PA4 (SW2) koppelen aan EXTI 4.
SYSCFG->EXTICR[1] |= SYSCFG_EXTICR2_EXTI4_PA;

// Falling edge detecteren.
EXTI->FTSR = EXTI->FTSR | EXTI_FTSR_TR4;

// Interrupt toelaten.
EXTI->IMR = EXTI->IMR | EXTI_IMR_MR4;

// Eén van de 4 prioriteiten kiezen. Als er twee
// interrupts zijn met dezelfde prioriteit, wordt
// eerst diegene afgehandeld die de laagste
// 'position' heeft (zie vector table NVIC in datasheet).
NVIC_SetPriority(EXTI4_15_IRQn,0);

// Interrupt effectief toelaten.
NVIC_EnableIRQ(EXTI4_15_IRQn);
```

*Figuur 35: codefragment instellen externe interrupt voor SW2*

De interrupt wordt afgehandeld in deze interrupt handler:

```
// Interrupt handler aanmaken. De definities van
// de handler kan je vinden in startup_stp32f091xc.s.
void EXTI4_15_IRQHandler(void)
{
    // Als het een interrupt is van PA4, de LED's inschakelen.
    if((EXTI->PR & EXTI_PR_PR4) == EXTI_PR_PR4)
    {
        // Interrupt (pending) vlag wissen door
        // een 1 te schrijven...
        EXTI->PR |= EXTI_PR_PR4;

        // LED's inschakelen als bewijs van interrupt.
        GPIOC->ODR = GPIOC->ODR | GPIO_ODR_0;
        GPIOB->ODR = GPIOB->ODR | GPIO_ODR_3;
        GPIOB->ODR = GPIOB->ODR | GPIO_ODR_5;
        GPIOB->ODR = GPIOB->ODR | GPIO_ODR_4;
        GPIOB->ODR = GPIOB->ODR | GPIO_ODR_10;
        GPIOA->ODR = GPIOA->ODR | GPIO_ODR_8;
        GPIOC->ODR = GPIOC->ODR | GPIO_ODR_7;
        GPIOB->ODR = GPIOB->ODR | GPIO_ODR_6;
    }
}
```

*Figuur 36: codefragment afhandelen externe interrupt voor SW2*

### Vragen:

- Neem de reference manual datasheet bij de hand en zoek alle info op van de gebruikte registers in bovenstaande codefragmenten. Verklaar ook wat de C-code precies doet.

## 8 Strings, structs en pointers in C

In dit hoofdstuk bekijken we hoe we een string kunnen opbouwen in C, wat minder vanzelfsprekend is dan in andere programmeertalen.

Daarnaast gaan we dieper in op de syntax *perifერიენაამ*->*registernaam* die we al veelvuldig zijn tegengekomen in de voorgaande hoofdstukken zoals *RCC*->*AHBENR*, *GPIOC*->*ODR*, *RCC*->*CR* enz. Wat betekent die *perifერიენაამ*, het symbool -> en *registernaam* technisch gezien in C-code?

### 8.1 Strings

Het is belangrijk om te weten dat C geen ingebouwd **string-datatype** heeft, zoals andere programmeertalen. Strings worden echter voorgesteld als een array van karakters met een afsluitend nulkarakter '\0'.

Als dit laatste karakter wordt vergeten, werken sommige string-bibliotheekfuncties in C niet correct, omdat ze niet weten waar de karakterarray eindigt.

Om een string te maken, dienen we gebruik te maken van een karakterarray. Het declareren en vervolgens initialiseren gebeurt als volgt voor het woord "Test" dat vier karakters telt:

```
char tekst[5];
tekst[0] = 'T';
tekst[1] = 'e';
tekst[2] = 's';
tekst[3] = 't';
tekst[4] = '\0';
```

*Figuur 37: declaratie en initialisatie karakterarray in meerdere stappen*

De declaratie en initialisatie kan ook in één stap gebeuren, hier mag eventueel de opgegeven arraygrootte tussen [ ] weggelaten worden vermits de compiler dit automatisch zal tellen:

```
char tekst[] = { 'T', 'e', 's', 't', '\0' };
```

Of wat eenvoudiger:

```
char tekst[] = "Test";
```

*Figuur 38: declaratie en initialisatie karakterarray in één stap*

Opmerking:

In plaats van *char* kunnen we ook *int8\_t* of zelfs *uint8\_t* gebruiken. Het laatste maakt het mogelijk om ook de uitgebreide ASCII-reeks (128-255) te gebruiken.

### 8.2 Structs

Een struct is een complex datatype dat meerdere variabelen groepeerd onder één naam. De verschillende onderdelen uit een struct kunnen via één overkoepelende naam bereikt worden.

Het voordeel van een struct is vooral dat de programmeur zijn of haar data op een meer logische manier kan structureren. C ondersteunt immers geen klassen en objecten, dus structs zijn de meest geavanceerde datastructuur die beschikbaar is.

Een codevoorbeeld waar we zelf een struct opbouwen:

```
#if !defined(PERSOON_DEFINED)
#define PERSON_DEFINED

struct persoon {
    char voornaam[50];
    char achternaam[50];
    float lengte;
};
#endif
```

*Figuur 39: definiëren van een struct*

Bovenstaande code kan eventueel via een header-bestand (met extensie .h) toegevoegd worden aan het project. Op die manier blijft alles overzichtelijk en gestructureerd.

Een goede gewoonte is om een header-bestand steeds te laten starten met preprocessor-directief ***#if !defined(NAAM)*** of ***#ifndef NAAM*** waarbij *NAAM* uniek is over het ganse project.

Bv. neem de naam van het bestand en plaats dit in hoofdletters (soms neemt men in de naam ook de h-extensie op, bv. *PERSON\_H*).

Dit zorgt ervoor dat als het header-bestand in meerdere bestanden wordt opgenomen, de compiler niet meerdere keren dezelfde struct definieert, anders kan dit compilatiefouten opleveren.

Het nieuw aangemaakt type *persoon* kan nu gebruikt worden in het hoofdprogramma door bijvoorbeeld de variabele *mezelf* te declareren:

```
struct persoon mezelf;
```

Opmerking: overal waar we een variabele van een struct-type willen aanmaken, moeten we in C steeds *struct* voor dat type plaatsen. Dit kan worden vereenvoudigd d.m.v. *typedef* in de struct-definitie. We mogen structnaam *persoon* eventueel zelfs weglaten (het is dan een anonieme struct).

```
#if !defined(PERSOON_DEFINED)
#define PERSON_DEFINED

typedef struct {
    char voornaam[50];
    char achternaam[50];
    float lengte;
} Persoon;
#endif
```

*Figuur 40: definiëren van een struct via een typedef*

Nu hoeft er geen *struct* meer te staan bij de declaratie en kunnen we meteen *Persoon* als type gebruiken:

```
Persoon mezelf;
```

Het opvullen van de velden in *Persoon* gebeurt door de variabelenaam *mezelf* op te geven gevolgd door een punt en de veldnaam:

```
//Struct invullen met de gewenste data.  
sprintf(mezelf.voornaam, "Ludovic");  
sprintf(mezelf.achternaam, "Espeel");  
mezelf.lengte = 1.84;
```

Figuur 41: opvullen van de struct-velden

*sprintf* is een functie uit de C-bibliotheek *stdio.h* waarmee data kan gekopieerd worden naar een karakterarray. Of we kunnen ook de *strcpy*-functie gebruiken uit de C-bibliotheek *string.h*. Met *sprintf* is het ook mogelijk om variabelen daarin te verwerken, bijvoorbeeld:

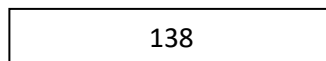
```
char buffer[100];  
int getal = 42;  
sprintf(buffer, "Het getal is %d", getal);
```

Figuur 42: voorbeeld *sprintf*-functie

### 8.3 Pointers

Ook in de C-taal worden variabelen opgeslagen op een bepaald adres in het RAM-geheugen. Veronderstel bijvoorbeeld een variabele *getal* dat 8-bit breed is (bv. *uint8\_t*), opgeslagen is op adres 12543 en als waarde 138 heeft. Dit kunnen we in het RAM-geheugen min of meer grafisch voorstellen door:

*getal* (adres = 12543)



Het adres van een variabele in C-taal kan opgevraagd worden via het **&**-teken (ampersand). Zo kunnen we stellen dat:

*&getal* gelijk moet zijn aan 12543.

In de C-taal bestaan er speciale types die **als inhoud een adres bevatten** naar een ander type. Zoiets heet men een **pointervariabele**. In C# kunnen we dit vergelijken met een referentietype.

Om een pointervariabele te maken gebruiken we het \*-teken (asterisk). Als we een pointer naar een unsigned 8-bit getal willen maken, kunnen we dit als volgt doen:

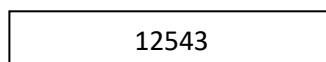
```
uint8_t *getalPointer;
```

Om daarin dan het adres van een dergelijke 8-bit variabele te plaatsen kunnen we:

```
getalPointer = &getal;
```

Als we dit via een grafisch in het RAM-geheugen willen voorstellen:

*getalPointer* (adres = 14592)



Daarna kunnen we *getalPointer* gebruiken op **twee manieren**:

1. We kunnen de **inhoud van *getalPointer*** opvragen. Die inhoud is dus **het adres van *getal* (12543)**. Dit doen we zo:  
`... = getalPointer;`
2. We kunnen de **inhoud waarnaar *getalPointer* wijst** opvragen. Die inhoud is de **inhoud van *getal* (138)**. Dit noemt men ook vaak **dereferencen van de pointer**.  
`... = *getalPointer;`

Het bovenstaande is wel verwarrend. Waarom? Omdat het **\*-teken** dus **twee betekenissen** heeft! We kunnen het gebruiken bij de aanmaak van een variabele, dan creëren we dus een **pointervariabele**.

Maar we kunnen het ook gebruiken bij een bestaande pointervariabele, dan krijgen we de **data waarnaar de pointervariabele verwijst**.

Probeer je dit goed voor te stellen! Maak steeds een tekening van hoe de variabelen zich in het RAM-geheugen gedragen! Enkel dan kan je pointers echt goed begrijpen.

Hieronder volgt een codevoorbeeld met pointers:

```
25 uint8_t getal = 138;
26 uint8_t* getalPointer;

47     sprintf(text, "Waarde van getal: %d.\r\n", getal);
48     StringToUsart2(text);
49
50     sprintf(text, "Adres van getal: %d.\r\n", (uint32_t)&getal);
51     StringToUsart2(text);
52
53     getalPointer = &getal;
54
55     sprintf(text, "Waarde van getalPointer: %d.\r\n", (uint32_t)getalPointer);
56     StringToUsart2(text);
57
58     sprintf(text, "Adres van getalPointer: %d.\r\n", (uint32_t)&getalPointer);
59     StringToUsart2(text);
60
61     sprintf(text, "Waarde naar waar getalPointer wijst: %d.\r\n", *getalPointer);
62     StringToUsart2(text);
63
64     WaitForMs(5000);
65
66     // Pas de waarde van getal aan via getalPointer...
67     *getalPointer = 139;
```

*Figuur 43: codevoorbeeld met pointers*

Het **\*-teken** kan dus inderdaad twee betekenissen hebben. Ofwel wordt het gebruikt om een pointer naar een bepaald type aan te maken (lijn 26), ofwel voor het opvragen of aanpassen van de data die aan een bepaalde pointervariabele is gekoppeld (lijnen 61 en 67).

Het **&-teken** wordt gebruikt om van een bepaalde variabele het adres op te vragen (lijnen 50, 53 en 58).



Als we voorgaande code uitvoeren, krijgen we dit te zien op de computer in bv. Putty:

```
COM7 - PuTTY
Waarde van getal: 138.
Adres van getal: 536870916.
Waarde van getalPointer: 536870916.
Adres van getalPointer: 536871016.
Waarde naar waar getalPointer wijst: 138.
```

Figuur 44: uitvoer van het codevoorbeeld met pointers

Naslagwerk:

Doorneem zelf nog eens de bijlage *Essential C* voor meer info over strings, structs en pointers. Dit vind je terug in *Section 3: Complex Data Types* op p. 15-23 en in *Section 4: Functions* op p. 26-28.

## 8.4 CMSIS-structuur

De CMSIS-bibliotheek maakt volop gebruik van structs en pointers om het programmeren gemakkelijker te maken.

Om de **samenwerking tussen structs en pointer** beter te begrijpen, zullen we eerst onderstaand voorbeeld bekijken: we maken een domoticasysteem voor een huis met twee verdiepingen en op die twee verdiepingen zijn telkens 4 lampen voorzien.

```
16 typedef struct{
17     // Hier stellen de 4 LSB's de 4 lampen voor op het gelijkvloers.
18     // De 4 MSB's zijn 'reserved' en moeten steeds op nul staan.
19     uint8_t verlichtingGelijkvloers;
20
21     // Hier stellen de 4 LSB's de 4 lampen voor op de eerste verdieping.
22     // De 4 MSB's zijn 'reserved' en moeten steeds op nul staan.
23     uint8_t verlichtingEersteVerdieping;
24 }DomoticaSysteem;
25
26
27 DomoticaSysteem huisVanPapaEnMama;    // ;-)
28 DomoticaSysteem* huisVanDeKindjes;    // :-p
29
30
31 // Schakel alle vier de lichten op het gelijkvloers aan.
32 huisVanPapaEnMama.verlichtingGelijkvloers = 0x0F;
33 // Schakel alle lichten boven uit.
34 huisVanPapaEnMama.verlichtingEersteVerdieping = 0x00;
35
36 // Omdat de kindjes nog thuis wonen, kan je bijvoorbeeld via
37 // een pointer naar 'huisVanPapaEnMama' ook diezelfde lichten
38 // schakelen... Let op: het gaat dus wel degelijk over dezelfde
39 // lichtpunten! Je schakelt ze gewoon via een andere variabele.
40 huisVanDeKindjes = &huisVanPapaEnMama;
41 // Schakel alle lichten op het gelijkvloers uit.
42 // Merk op dat je nu geen '.' meer mag gebruiken om binnen te
43 // treden in de struct, maar een '->'!!! Verder blijft de
44 // redenering hetzelfde.
45 huisVanDeKindjes->verlichtingGelijkvloers = 0x00;
46 // Schakel 1 licht aan op de eerste verdieping...
47 huisVanDeKindjes->verlichtingEersteVerdieping |= 0x01;
```

Figuur 45: codevoorbeeld met struct en pointers

In het voorgaand codevoorbeeld bevat de pointervariabele *huisVanDeKindjes* het adres van de struct *DomoticaSysteem*.

Om de velden te kunnen lezen of schrijven dienen we **eerst** de inhoud van de pointervariabele op te vragen (*dereferencen*), vandaar dat dit tussen haakjes staat. Vervolgens kunnen we aan het veld: *(\*huisVandeKindjes).verlichtingGelijkvloers = 0x00;*

Vaak vindt men deze syntax wat omslachtig, een elegantere manier is d.m.v. de **-> operator**: *huisVandeKindjes->verlichtingGelijkvloers = 0x00;*

Als het voorgaande wat duidelijk is geworden, dan kunnen we zonder twijfel de CMSIS structs en pointers ook begrijpen. Bekijk eens in Keil de inhoud van het bestand *stm32f091xc.h* en speur eens rond naar o.a. deze stukken code:

```
683 | #define RCC_BASE                (AHBPERIPH_BASE + 0x00001000)
763 | #define RCC                      ((RCC_TypeDef *) RCC_BASE)
463 | typedef struct
464 | {
465 |     __IO uint32_t CR;
466 |     __IO uint32_t CFGR;
467 |     __IO uint32_t CIR;
468 |     __IO uint32_t APB2RSTR;
469 |     __IO uint32_t APB1RSTR;
470 |     __IO uint32_t AHBENR;
471 |     __IO uint32_t APB2ENR;
472 |     __IO uint32_t APB1ENR;
473 |     __IO uint32_t BDCR;
474 |     __IO uint32_t CSR;
475 |     __IO uint32_t AHBSTR;
476 |     __IO uint32_t CFGR2;
477 |     __IO uint32_t CFGR3;
478 |     __IO uint32_t CR2;
479 | } RCC_TypeDef;
```

```
// Clock voor GPIOA inschakelen (on board LED).
RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;
```

Figuur 46: gebruikmakend van de CMSIS-structs en pointers

Opmerkingen:

- Het kan zijn dat de regelnummers hier niet (meer) overeenkomen met ons project in Keil, er gebeuren soms updates aan de CMSIS-bibliotheken...
- Op lijn 763 gaan we een 32-bitwaarde *casten* (omzetten) naar een *RCC\_TypeDef* pointer en definiëren als *RCC*.

Vragen:

- Bekijk eens hoe bv. *GPIOC* is opgebouwd in het bestand *stm32f091xc.h*.